

Estándares de Desarrollo: Plataforma DataForge IA

1. Introducción

El presente documento establece los **Estándares de Desarrollo** para el proyecto **DataForge IA**, una plataforma web de análisis predictivo para PYMES. El objetivo es asegurar la calidad, consistencia, mantenibilidad y escalabilidad del código base, facilitando la colaboración entre los miembros del equipo y la integración de nuevos desarrolladores. La adhesión a estos estándares es obligatoria para todos los participantes en el ciclo de vida del desarrollo de software.

2. Convenciones de Código

La consistencia en el estilo de codificación es fundamental para la legibilidad y mantenibilidad del software. Se adoptarán las siguientes convenciones:

2.1. Nomenclatura

- **Variables y Funciones:** `camelCase` (ej. `nombreUsuario`, `calcularTotal()`).
- **Clases e Interfaces:** `PascalCase` (ej. `UserService`, `IDataProcessor`).
- **Constantes:** `UPPER_SNAKE_CASE` (ej. `MAX_RETRIES`, `API_KEY`).
- **Archivos:** `snake_case.ext` para Python (ej. `user_service.py`), `kebab-case.ext` para JavaScript/Frontend (ej. `user-list.component.tsx`).

2.2. Formato del Código

- **Indentación:** 4 espacios para Python, 2 espacios para JavaScript/TypeScript. No se usarán tabulaciones.
- **Longitud de Línea:** Máximo 120 caracteres por línea para mejorar la legibilidad en diferentes entornos.
- **Espacios:** Espacios alrededor de operadores, después de comas y antes de llaves de apertura.
- **Salto de Línea:** Un salto de línea al final de cada archivo. Dos saltos de línea entre definiciones de funciones/clases de alto nivel.

2.3. Comentarios

- **Comentarios de Bloque:** Utilizar para explicar la lógica compleja, algoritmos no triviales o secciones de código que no son autoexplicativas. Deben ser concisos y relevantes.
- **Docstrings (Python):** Todas las funciones, clases y módulos deben tener docstrings que describan su propósito, parámetros, valores de retorno y excepciones. Se seguirá el formato Google Style Docstrings [1].
- **JSDoc (JavaScript/TypeScript):** Similar a docstrings, para documentar funciones, clases y tipos en el frontend.

2.4. Estándares Específicos por Lenguaje

- **Python:** Se seguirá el estándar **PEP 8** para el estilo de código. Se utilizarán herramientas como `Flake8` o `Black` para la verificación y formateo automático [2].
- **JavaScript/TypeScript:** Se utilizará `ESLint` con una configuración estándar (ej. Airbnb o Google Style) y `Prettier` para el formateo automático. Para TypeScript, se seguirán las guías de estilo recomendadas por la comunidad.

3. Buenas Prácticas de Desarrollo

La adopción de buenas prácticas es esencial para construir un software robusto, seguro y de alto rendimiento.

3.1. Principios SOLID

Se aplicarán los principios SOLID (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) para diseñar clases y módulos con alta cohesión y bajo acoplamiento [3].

3.2. Pruebas Unitarias e Integración

- **Cobertura de Pruebas:** Se buscará una cobertura de pruebas mínima del 80% para el código de negocio crítico. Todas las funcionalidades nuevas deben ir acompañadas de sus respectivas pruebas.
- **Frameworks:** `Pytest` para Python, `Jest` o `React Testing Library` para JavaScript/TypeScript.
- **CI/CD:** Las pruebas unitarias y de integración se ejecutarán automáticamente en cada push a las ramas de desarrollo y en cada pull request.

3.3. Seguridad

- **Validación de Entradas:** Todas las entradas de usuario deben ser validadas y sanitizadas para prevenir ataques como inyección SQL, XSS, etc.
- **Autenticación y Autorización:** Implementar mecanismos robustos de autenticación (ej. JWT) y autorización basada en roles y permisos.
- **Cifrado:** Utilizar cifrado para datos sensibles en tránsito (HTTPS) y en reposo (cifrado de base de datos/almacenamiento de objetos).
- **Gestión de Secretos:** Las claves API, credenciales de base de datos y otros secretos no deben estar hardcodeados en el código. Utilizar variables de entorno o servicios de gestión de secretos.

3.4. Rendimiento y Escalabilidad

- **Optimización de Consultas:** Optimizar las consultas a la base de datos para reducir los tiempos de respuesta.
- **Manejo Asíncrono:** Utilizar patrones asíncronos para operaciones de I/O intensivas o de larga duración (ej. entrenamiento de modelos de IA).
- **Manejo de Errores:** Implementar un manejo de errores robusto y logging adecuado para facilitar la depuración y el monitoreo.

4. Uso de Control de Versiones (Git)

Git será la herramienta principal para el control de versiones. Se seguirá un flujo de trabajo basado en ramas (ej. Git Flow o GitHub Flow) para gestionar el desarrollo.

4.1. Ramas

- **main (o master)**: Rama principal, siempre estable y lista para producción.
- **develop**: Rama de integración para nuevas funcionalidades. Todas las ramas de características se fusionan aquí.
- **feature/<nombre-característica>**: Ramas para el desarrollo de nuevas funcionalidades. Deben ser cortas y específicas.
- **bugfix/<id-bug>**: Ramas para la corrección de errores.
- **release/<version>**: Ramas para preparar una nueva versión de producción.

4.2. Commits

- **Mensajes de Commit:** Deben ser claros, concisos y descriptivos. Seguir la convención Tipo(Ámbito): Descripción (ej. `feat(auth):` Añadir registro de usuarios, `fix(db):` Corregir migración de esquema).
- **Atomicidad:** Cada commit debe representar un cambio lógico único y completo.

4.3. Pull Requests (PRs)

- **Revisión de Código:** Todas las fusiones a `develop` o `main` deben realizarse a través de Pull Requests y requerir al menos una revisión y aprobación por parte de otro miembro del equipo.
- **Pruebas Automatizadas:** Los PRs deben pasar todas las pruebas automatizadas (unitarias, integración) antes de poder ser fusionados.

5. Documentación del Código

La documentación es tan importante como el código mismo. Se mantendrá una cultura de documentación continua.

5.1. Documentación Interna (Código)

- **Docstrings/JSDoc:** Como se mencionó en la sección de Convenciones de Código, todas las funciones, clases y módulos críticos deben estar documentados.
- **Comentarios en Línea:** Para explicar decisiones de diseño complejas o trucos específicos del lenguaje.

5.2. Documentación Externa (Proyecto)

- **README.md:** Cada repositorio debe tener un archivo `README.md` que describa el propósito del proyecto, cómo configurarlo, cómo ejecutarlo y cómo contribuir.
- **Wiki/Confluence:** Para documentación de alto nivel, decisiones de arquitectura, guías de despliegue, manuales de usuario y FAQs.
- **Diagramas:** Mantener actualizados los diagramas UML (clases, componentes, ER) y de arquitectura.

6. Conclusiones

La implementación y el seguimiento riguroso de estos Estándares de Desarrollo son fundamentales para el éxito a largo plazo de DataForge IA. Al adherirse a estas directrices, el equipo de desarrollo construirá un producto de alta calidad, mantenible y escalable, que podrá evolucionar eficientemente para satisfacer las necesidades cambiantes del mercado y de las PYMES. Estos estándares fomentan una cultura de excelencia técnica y colaboración efectiva.

7. Referencias

[1] Google. (n.d.). *Google Python Style Guide*. Retrieved from <https://google.github.io/styleguide/pyguide.html> [2] Python Software Foundation. (2001). *PEP 8 – Style Guide for Python Code*. Retrieved from <https://www.python.org/dev/peps/pep-0008/> [3] Martin, R. C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.

- Documento de Visión del Sistema: DataForge IA.
- Especificación de Requisitos de Software (SRS): DataForge IA.
- Documento de Casos de Uso: DataForge IA.
- Historias de Usuario: DataForge IA.
- Análisis de Stakeholders: DataForge IA.
- Documento de Levantamiento de Requerimientos: DataForge IA.
- Estudio de Factibilidad: DataForge IA.
- Análisis del Problema: DataForge IA.
- Diseño de Arquitectura de Software: DataForge IA.
- Diseño de Clases UML: DataForge IA.
- Diseño de Componentes: DataForge IA.
- Plan de Desarrollo Ágil: DataForge IA.
- Product Backlog: DataForge IA.